

ĐÁP ÁN — LẬP TRÌNH PHÂN TÍCH DỮ LIỆU

Đề số 02 · Kiểm tra thực hành 60 phút · Buổi 1–5 · Dữ liệu sinh với SEED = 2502

Nguyên tắc chấm chung

- Chấm theo **kết quả và phương pháp**. Code viết khác đáp án nhưng ra đúng kết quả vẫn được đủ điểm.
- Không trừ điểm dây chuyền**: nếu câu trước sai (vd. C2) làm số liệu câu sau lệch, câu sau vẫn được đủ điểm khi phương pháp đúng trên dữ liệu của chính sinh viên.
- Chấp nhận sai lệch làm tròn ở chữ số cuối ($\pm 0,1$ triệu đồng; $\pm 0,001$ với tỉ lệ và độ lệch).
- Ô code báo lỗi khi Restart & Run All: không chấm phần kết quả của ô đó.
- Câu trả lời bằng lời (A2, D2, E) phải gắn với bối cảnh bài toán, không chỉ nêu con số.
- Số liệu trong các bảng dưới đây in đúng như notebook `dap_an/dap_an.ipynb` (dấu chấm thập phân).

Phần	Nội dung	Câu	Điểm
A	Python thuần: tập tin văn bản, regex, hàm	A1, A2	2,0
B	NumPy & lập trình hướng đối tượng	B1, B2, B3	2,0
C	Pandas: đọc, làm sạch, tích hợp dữ liệu	C1, C2, C3	2,5
D	Biến đổi & thu gọn dữ liệu	D1, D2	1,5
E	Trực quan hoá	E	2,0
Tổng			10,0

Phần A — Python thuần: tập tin văn bản, regex, hàm

2,0 điểm

Câu A1. Lọc dòng hợp lệ bằng regex

1,0 điểm

Chỉ số	Kết quả chuẩn
Số dòng hợp lệ	754
Số dòng không hợp lệ	48 (6 kiểu lỗi $\times$ 8 dòng)
Số sự kiện theo trạm	DN01 132 · DN02 135 · DN03 119 · DN04 122 · HA01 136 · HA02 110
<code>dong_loi.txt</code>	48 dòng
Dòng bỏ qua	7 dòng bắt đầu bằng # · 12 dòng trống

```
MAU = re.compile(r'(\d{4}-\d{2}-\d{2}) (\d{2}:\d{2}) \ | ((?:DN|HA)\d{2}) \ | (X[DM]\d{3}) '
                r'\ | (MUON|TRA) \ | pin=(\d{1,3})%')
hop_le, loi = [], []
with open(DATA_DIR / 'nhat_ky_khoa_xe.txt', encoding='utf-8') as f:
    for dong in f:
        dong = dong.rstrip('\n')
        if dong.strip() == '' or dong.startswith('#'):
            continue
```

```
(hop_le if MAU.fullmatch(dong) else loi).append(dong)
with open('dong_loi.txt', 'w', encoding='utf-8') as f:
    f.write('\n'.join(loi) + '\n')
dem_tram = Counter(MAU.fullmatch(d).group(3) for d in hop_le)
print(len(hop_le), len(loi), dict(sorted(dem_tram.items())))
```

Tiêu chí	Điểm
Mẫu regex đúng: thoát ký tự , (DN HA)\d{2}, X[DM]\d{3}, nhóm MUON/TRA, pin=\d{1,3}%	0,3
Khớp toàn dòng (fullmatch hoặc ^...\$)	0,1
Bỏ qua dòng trống và dòng #	0,1
Số dòng hợp lệ / không hợp lệ đúng	0,2
Đếm sự kiện theo trạm bằng Counter, in theo thứ tự mã trạm	0,1
Ghi dong_loi.txt đúng (utf-8, mỗi dòng lỗi một dòng)	0,2

Lỗi thường gặp: Dùng re.match hoặc re.search với mẫu không có \$ → 8 dòng thừa trường | gps=off ở cuối lọt vào, ra **762** dòng hợp lệ và 40 dòng lỗi — mất 0,1 (khớp toàn dòng) và 0,2 (số dòng). Không thoát | → regex hiểu là phép “hoặc”: fullmatch ra 0 dòng hợp lệ, search khớp cả 802 dòng. Tính cả dòng trống và dòng # vào lỗi → ra 67 thay vì 48. Quên rstrip('\n') khi dùng fullmatch → 0 dòng hợp lệ. 6 kiểu lỗi cài sẵn: mã xe 2 chữ số (XD08) · sự kiện viết thường · thiếu trường pin · giờ 07h15 · trạm ngoài hệ thống QN01 · thừa trường cuối dòng.

Câu A2. Thống kê pin yếu

1,0 điểm

Xe	Lần trả	Pin yếu	Tỉ lệ	Xe	Lần trả	Pin yếu	Tỉ lệ
XD001	17	2	0.118	XD013	13	0	0.000
XD002	16	1	0.062	XD014	14	3	0.214
XD003	19	4	0.211	XM001	16	1	0.062
XD004	13	3	0.231	XM002	12	2	0.167
XD005	13	1	0.077	XM003	20	7	0.350
XD006	16	1	0.062	XM004	20	2	0.100
XD007	22	7	0.318	XM005	17	2	0.118
XD008	14	0	0.000	XM006	15	2	0.133
XD009	19	3	0.158	XM007	20	0	0.000
XD010	19	1	0.053	XM008	12	2	0.167
XD011	17	1	0.059	XM009	8	2	0.250
XD012	20	5	0.250				
XM010	5	4	0.800 — bị loại (< 8 lần trả)				

Xe có tỉ lệ pin yếu cao nhất: **XM003 — 0.350 (35,0%)**. Có sự kiện ở cả hai thành phố: **XD004, XD012, XM008, XM010**.

```
def ti_le_pin_yeu(cac_dong, nguong=20, toi_thieu=8):
    so_tra, so_yeu = Counter(), Counter()
    for d in cac_dong:
        _, _, _, xe, su_kien, pin = MAU.fullmatch(d).groups()
        if su_kien != 'TRA':
            continue
```

```

so_tra[xe] += 1
if int(pin) < nguong:
    so_yeu[xe] += 1
return {xe: so_yeu[xe] / n for xe, n in so_tra.items() if n >= toi_thieu}

kq = ti_le_pin_yeu(hop_le)
max(kq, key=kq.get) # 'XM003'

thanh_pho = {}
for d in hop_le:
    _, _, tram, xe, *_ = MAU.fullmatch(d).groups()
    thanh_pho.setdefault(xe, set()).add(tram[:2])
sorted(xe for xe, s in thanh_pho.items() if s == {'DN', 'HA'}) # XD004, XD012, XM008, XM010

```

Tiêu chí	Điểm
Hàm đúng tên, tham số, giá trị mặc định <code>nguong=20</code> , <code>toi_thieu=8</code>	0,15
Chỉ xét sự kiện TRA; đổi pin sang <code>int</code> và so sánh nhỏ hơn hẳn <code>nguong</code>	0,25
Đếm theo xe, giữ xe có $\geq$ <code>toi_thieu</code> lần trả, trả về dict	0,2
In tỉ lệ 3 chữ số thập phân, xác định đúng XM003	0,2
Dùng <code>set</code> tìm đúng 4 xe có sự kiện ở cả hai thành phố	0,2

Bắt: Dùng `<=` thay `<`: có 14 lần trả đúng 20%, riêng XD007 có 4 lần $\rightarrow$ XD007 vọt lên **0.500** và thành xe cao nhất — mất 0,25 (so sánh) và 0,2 (kết luận). Quên lọc TRA (đếm cả MUON): XM010 có 10 sự kiện nên lọt qua ngưỡng 8, tỉ lệ 0.400 và thành xe cao nhất — mất 0,25 và 0,2. Không áp dụng `toi_thieu`: XM010 có 0.800 nhưng chỉ 5 lần trả. So sánh chuỗi `pin < '20'` (quên `int`) $\rightarrow$ `'5' < '20'` là False, kết quả ra XD007 0.182 — mất 0,25. Dùng `> toi_thieu` làm mất XM009 (đúng 8 lần trả) — trừ 0,1. Câu thành phố xét **tất cả** dòng hợp lệ nên phải có XM010; thiếu XM010 trừ 0,1.

Phần B — NumPy & lập trình hướng đối tượng

2,0 điểm

Câu B1. Ma trận lượt thuê

0,5 điểm

Chỉ số	Kết quả chuẩn
Tổng lượt theo trạm	DN01 215 · DN02 211 · DN03 209 · DN04 123 · HA01 219 · HA02 170 $\rightarrow$ cao nhất HA01
Trung bình theo khung giờ	05–08h 31.33 · 08–11h 34.33 · 11–14h 29.67 · 14–17h 36.33 · 17–20h 59.50 $\rightarrow$ cao nhất 17–20h
Tỉ trọng giờ cao điểm	DN01 0.526 · DN02 0.588 · DN03 0.368 · DN04 0.447 · HA01 0.434 · HA02 0.476 $\rightarrow$ lớn hơn 0,5: DN01, DN02
Số ô lớn hơn TB của chính khung giờ	14

```

tong_tram = L.sum(axis=1); TRAM[tong_tram.argmax()] # 'HA01'
tb_khung = L.mean(axis=0); KHUNG[tb_khung.argmax()] # '17–20h'
ty_trong = L / L.sum(axis=1, keepdims=True) # (6, 5) / (6, 1)
cao_diem = ty_trong[:, [0, 4]].sum(axis=1) # chọn cột 0 và cột 4
np.array(TRAM)[cao_diem > 0.5] # ['DN01', 'DN02']
(L > L.mean(axis=0)).sum() # 14 – (6, 5) so với (5,)

```

Tiêu chí	Điểm
Đúng <code>axis</code> cho tổng/trung bình và dùng <code>argmax</code>	0,2
Tỉ trọng bằng broadcasting, chọn đúng 2 cột không liền nhau, lọc trạm bằng mặt nạ boolean	0,15
Mặt nạ so với trung bình từng cột , ra 14	0,15

Lỗi thường gặp: So với trung bình của từng trạm (`axis=1`) ra 12, so với trung bình cả ma trận ra 13 → mất 0,15. Viết lát cắt `ty_trong[:, 0:4]` thay vì chọn cột `[:, [0, 4]]` → cộng nhầm 4 khung đầu (`[:, [0, -1]]` hay `[:, ::4]` vẫn đúng). Chia (6, 5) cho `L.sum(axis=1)` shape (6,) → `ValueError broadcasting`. Tổng DN01 (215) sát HA01 (219): đọc nhầm dòng/cột sẽ ra DN01.

Câu B2. Giải hệ phương trình

0,5 điểm

Phí mở khoá **5 nghìn** · giá mỗi phút **0,6 nghìn** · phụ phí mỗi km **2 nghìn**. $\det(A) = -60 \neq 0$ nên hệ có nghiệm duy nhất. Dự đoán chuyến 30 phút, 6 km: $5 + 0,6 \times 30 + 2 \times 6 = 35$ nghìn đồng.

```
A = np.array([[1, 20, 3], [1, 35, 8], [1, 50, 9]]) # cột 1 là hệ số của phí mở khoá
b = np.array([23, 42, 53])
x = np.linalg.solve(A, b) # [5. 0.6 2. ]
np.allclose(A @ x, b) # True
np.array([1, 30, 6]) @ x # 35.0
```

Tiêu chí	Điểm
Lập đúng ma trận hệ số A (3×3, có cột hệ số 1 cho phí mở khoá) và vector b	0,15
<code>np.linalg.solve</code> ra đúng nghiệm	0,2
Kiểm tra lại bằng <code>A @ x</code> (hoặc <code>np.dot</code>) và <code>np.allclose</code>	0,1
Dự đoán đúng 35 nghìn đồng bằng tích vô hướng với nghiệm	0,05

Lỗi thường gặp: Bỏ cột hệ số 1 → ma trận 3×2, `np.linalg.solve` báo `LinAlgError` (ma trận không vuông) — không có điểm hai tiêu chí đầu. Dự đoán bằng `[30, 6]` quên phí mở khoá → 30 nghìn, mất 0,05. Dùng `np.linalg.inv(A) @ b` vẫn đúng kết quả — cho đủ điểm.

Câu B3. Lớp chuyển đi

1,0 điểm

CD001: 25 phút · 6.0 km · 14.4 km/h · cước 17,500đ
 CD002: 48 phút · 10.4 km · 13.0 km/h · cước 9,000đ
 Lỗi: CD001: thời lượng 0 phút không hợp lệ

```
class ChuyenDi:
    PHI_MO_KHOA = 5000

    def __init__(self, ma, phut, km, gia_phut):
        self.ma, self.km, self.gia_phut = ma, km, gia_phut
        self.phut = phut # đi qua setter

    @property
    def phut(self):
        return self._phut

    @phut.setter
    def phut(self, gia_tri):
        if gia_tri <= 0:
            raise ValueError(f'{self.ma}: thời lượng {gia_tri} phút không hợp lệ')
```

```

        self._phut = gia_tri

    def cuoc_phi(self):
        return self.PHI_MO_KHOA + self.phut * self.gia_phut

    def toc_do_tb(self):
        return self.km / (self.phut / 60)

    def __str__(self):
        return (f'{self.ma}: {self.phut} phút · {self.km:.1f} km · '
                f'{self.toc_do_tb():.1f} km/h · cước {self.cuoc_phi():.0f}đ')

class ChuyenDiGoiThang(ChuyenDi):
    def __init__(self, ma, phut, km, gia_phut, phut_mien_phi=30):
        super().__init__(ma, phut, km, gia_phut)
        self.phut_mien_phi = phut_mien_phi

    def cuoc_phi(self):
        return max(0, self.phut - self.phut_mien_phi) * self.gia_phut

```

Tiêu chí	Điểm
@property + @phut.setter ném ValueError; kiểm tra có hiệu lực cả lúc khởi tạo	0,35
Thuộc tính lớp PHI_MO_KHOA; cuoc_phi, toc_do_tb, __str__ đúng định dạng mẫu	0,25
Lớp con dùng super().__init__, ghi đè cuoc_phi có max(0, ...) — __str__ của lớp cha tự dùng cước mới	0,25
Ba lệnh kiểm thử in đúng, bắt lỗi bằng try/except	0,15

Lỗi thường gặp: Trong __init__ gán self._phut = phut trực tiếp → bỏ qua kiểm tra, trừ 0,15. Getter trả self.phut → RecursionError. Gán self.ma sau self.phut → setter báo AttributeError khi tạo thông báo lỗi. Thiếu max(0, ...) : chuyển gói tháng 20 phút ra cước âm (−5,000đ) — kiểm thử CD002 không lộ lỗi nhưng trừ 0,1. Viết cứng 5000 trong cuoc_phi thay vì dùng thuộc tính lớp — trừ 0,05. Lớp con viết lại __str__ thay vì chỉ ghi đè cuoc_phi vẫn ra đúng chuỗi — cho 0,15/0,25 vì không tận dụng đa hình.

Phần C — Pandas: đọc, làm sạch và tích hợp dữ liệu

2,5 điểm

Câu C1. Đọc dữ liệu từ ba loại nguồn

0,5 điểm

Chỉ số	Kết quả chuẩn
chuyen_di.shape	(2635, 9)
khach_hang.shape · goi_cuoc.shape	(508, 5) · (3, 2)
tram.shape · loại_xe.shape	(6, 4) · (3, 5)
Giá trị thiếu trong chuyen_di	ket_thuc 48 · ma_kh 913 · quang_duong_km 40 · danh_gia 559

```

cd = pd.read_csv(DATA_DIR / 'chuyen_di.csv')
for cot in ['bat_dau', 'ket_thuc']:
    cd[cot] = pd.to_datetime(cd[cot], format='%d/%m/%Y %H:%M')
kh = pd.read_excel(DATA_DIR / 'khach_hang.xlsx', sheet_name='khach_hang')
goi = pd.read_excel(DATA_DIR / 'khach_hang.xlsx', sheet_name='goi_cuoc')
with sqlite3.connect(DATA_DIR / 'he_thong.db') as con:
    tram = pd.read_sql('SELECT * FROM tram', con)

```

```
loai = pd.read_sql('SELECT * FROM loai_xe', con)
cd.isna().sum()[lambda s: s > 0]
```

Tiêu chí	Điểm
read_csv + to_datetime cho cả hai cột với format hoặc dayfirst=True	0,15
Đọc đúng cả hai sheet (sheet_name)	0,1
sqlite3.connect + pd.read_sql với câu lệnh SELECT cho 2 bảng	0,15
In shape 5 bảng và số giá trị thiếu	0,1

Lỗi thường gặp: pd.to_datetime không chỉ định định dạng → pandas đoán tháng/ngày từ dòng đầu (01/03/2025) rồi báo ValueError ở “13/03/2025 07:15”. Dùng format='mixed' thì không báo lỗi nhưng đảo ngày–tháng với mọi ngày ≤ 12: dữ liệu rải khắp tháng 1–12 và biểu đồ Phần E sai hình. Trừ 0,15. Chỉ chuyển bat_dau → bước 3 của C2 báo TypeError khi trừ chuỗi.

Câu C2. Làm sạch bảng chuyển đi

1,0 điểm

Chỉ số	Kết quả chuẩn
Sau bước 1 (bỏ trùng)	2600 dòng — bỏ 35
Sau bước 2	Từ 15 cách viết còn đúng 3 giá trị
Bước 3	9 giá trị phut ≤ 0 thành NaN (cùng 48 chuyển trống ket_thuc → 57 NaN) · bỏ 27 chuyển mở khoá thử → 2573 dòng
Bước 4	11 giá trị GPS lỗi thành NaN (5 âm, 6 lớn hơn 80)
Bước 5 — ngưỡng trên	Xe đạp 44.5 · Xe đạp điện 57.0 · Xe máy điện 96.0 phút
Bước 5 — ngoại lai	Xe đạp 31 · Xe đạp điện 37 · Xe máy điện 23
Số chuyển sau làm sạch	Xe đạp 881 · Xe đạp điện 976 · Xe máy điện 716
Trung vị phut theo loại xe	Xe đạp 17 · Xe đạp điện 24 · Xe máy điện 37

```
cd = cd.drop_duplicates() # 1
CHUAN = {'xe đạp': 'Xe đạp', 'xe dap': 'Xe đạp',
         'xe đạp điện': 'Xe đạp điện', 'xe dap dien': 'Xe đạp điện',
         'xe máy điện': 'Xe máy điện', 'xe may dien': 'Xe máy điện'}
cd['loai_xe'] = cd['loai_xe'].str.strip().str.lower().map(CHUAN) # 2
cd['phut'] = (cd['ket_thuc'] - cd['bat_dau']).dt.total_seconds() / 60 # 3
cd.loc[cd['phut'] <= 0, 'phut'] = np.nan
cd = cd[~(cd['phut'] < 2)] # giữ NaN
sai_gps = (cd['quang_duong_km'] < 0) | (cd['quang_duong_km'] > 80) # 4
cd.loc[sai_gps, 'quang_duong_km'] = np.nan
q1 = cd.groupby('loai_xe')['phut'].transform(lambda s: s.quantile(0.25)) # 5
q3 = cd.groupby('loai_xe')['phut'].transform(lambda s: s.quantile(0.75))
tren = q3 + 1.5 * (q3 - q1)
cd.loc[cd['phut'] > tren, 'loai_xe'].value_counts()
cd['phut'] = cd['phut'].clip(upper=tren)
for cot in ['phut', 'quang_duong_km']:
    cd[cot] = cd[cot].fillna(cd.groupby('loai_xe')[cot].transform('median'))
```

Tiêu chí	Điểm
Bước 1: drop_duplicates()	0,1
Bước 2: strip + lower + ánh xạ khớp nguyên chuỗi, cả bản không dấu; còn đúng 3 giá trị	0,2
Bước 3: phut bằng dt.total_seconds() / 60; ≤ 0 thành NaN; bỏ < 2 nhưng giữ NaN	0,25

Tiêu chí	Điểm
Bước 4: đổi giá trị âm và > 80 thành NaN	0,1
Bước 5: IQR theo nhóm (<code>groupby + transform</code>), <code>clip</code> về ngưỡng trên, điền trung vị theo loại xe cho cả hai cột	0,35

Bẫy: Lọc bằng `cd[cd['phut'] >= 2]` làm mất luôn 57 chuyến đang NaN → còn 2516 dòng thay vì 2573: bước 3 tối đa 0,1. Dùng `.dt.seconds` thay `total_seconds()`: khoảng thời gian âm bị quy thành 1352–1422 phút, 9 lỗi đồng hồ không bị bắt ở bước 3 mà thành “ngoại lai” ở bước 5 — trừ 0,1. Chuẩn hoá bằng `str.contains('đạp')` gộp luôn xe đạp điện vào xe đạp (1796 dòng thay vì 893): bước 2 không có điểm; chỉ `strip().str.capitalize()` còn 6 giá trị (bản không dấu tách riêng) — cũng không có điểm. IQR chung cả cột: ngưỡng 66,0 phút đánh dấu 103 chuyến xe máy điện (gần 15% loại này) chỉ vì xe máy điện vốn đi lâu hơn — bước 5 tối đa 0,1.

Câu C3. Tích hợp dữ liệu

1,0 điểm

Chỉ số	Kết quả chuẩn
gia_30_phut là cột dẫn xuất	<code>(loai['gia_30_phut'] == loai['phi_mo_khoa'] + 30 * loai['gia_phut']).all()</code> → True
Mã khách trùng	8 (3 dòng trùng hoàn toàn + 5 dòng chỉ khác cách viết số điện thoại) → còn 500 khách
Số điện thoại không đủ 10 chữ số	9
Khách theo gói	Tiêu chuẩn 261 · Sinh viên 156 · Thân thiết 83
Chuyến có tram_muon ngoài danh mục	24 (đều là DN07)
Số dòng df · chuyến Khách lẻ	2549 · 918 (gồm ô trống và mã khách không tồn tại)
Tổng doanh thu	55,701,300 VNĐ
Doanh thu theo thành phố (triệu)	Đà Nẵng 33.3 · Hội An 22.4
Doanh thu theo trạm (triệu)	Phố cổ 13.6 · Cầu Rồng 11.3 · Mỹ Khê 9.1 · An Bàng 8.8 · Chợ Hàn 8.0 · Sơn Trà 4.9

```
(loai['gia_30_phut'] == loai['phi_mo_khoa'] + 30 * loai['gia_phut']).all()      # True
loai = loai.drop(columns='gia_30_phut')

kh['ma_kh'].duplicated().sum()                                              # 8
kh = kh.drop_duplicates(subset='ma_kh')
kh['so_dien_thoai'] = (kh['so_dien_thoai'].str.replace(r'\D', '', regex=True)
                      .str.replace(r'^84', '0', regex=True))
(kh['so_dien_thoai'].str.len() != 10).sum()                                # 9
kh = kh.merge(goi, on='goi_cuoc', how='left')

cd = cd[cd['tram_muon'].isin(tram['id'])]                                    # bỏ 24
df = (cd.merge(loai, left_on='loai_xe', right_on='ten_loai', how='left')
      .merge(kh[['ma_kh', 'goi_cuoc', 'giam_gia']], on='ma_kh', how='left',
            validate='many_to_one')
      .merge(tram, left_on='tram_muon', right_on='id', how='left')
      .drop(columns=['ten_loai', 'id']))
df['goi_cuoc'] = df['goi_cuoc'].fillna('Khách lẻ')
df['giam_gia'] = df['giam_gia'].fillna(0)
df['cuoc'] = df['phi_mo_khoa'] + df['phut'] * df['gia_phut']
df['doanh_thu'] = df['cuoc'] * (1 - df['giam_gia'])
```

Tiêu chí	Điểm
Chứng minh cột dẫn xuất bằng phép so sánh trên mọi dòng rồi xoá	0,15
Khử trùng theo <code>ma_kh</code> , chuẩn hoá số điện thoại bằng regex, đếm đúng 9, ghép gói để có <code>giam_gia</code>	0,3
Loại 24 chuyển DN07; ghép bằng <code>left_on / right_on</code> , <code>how='left'</code> giữ đủ 2549 dòng; gán Khách lẻ và giảm giá 0	0,3
<code>cuoc</code> , <code>doanh_thu</code> , tổng và doanh thu theo thành phố, theo trạm đúng	0,25

Đối chiếu nhanh tổng doanh thu sai — mỗi con số chỉ ra đúng một lỗi ở phía trước:

Sinh viên ra	Nguyên nhân
56,460,570 (2583 dòng)	Không bỏ dòng trùng (C2 bước 1)
54,521,960 (2492 dòng)	Lọc <code>phut >= 2</code> làm mất các chuyến đang trống <code>phut</code> (C2 bước 3)
56,551,040	Không winsorize ngoại lai (C2 bước 5)
54,371,880	Tính IQR chung cả cột thay vì theo loại xe (C2 bước 5)
55,505,540	Điền trung vị chung thay vì theo loại xe (C2 bước 5)
55,955,650 (2565 dòng)	Khử trùng khách bằng <code>drop_duplicates()</code> không có <code>subset</code> → chỉ bỏ được 3, 5 khách còn trùng nhân đôi 16 chuyến (C3). <code>validate='many_to_one'</code> sẽ báo lỗi ngay
33,737,600 (1631 dòng)	<code>how='inner'</code> khi ghép khách hàng — mất toàn bộ khách lẻ
58,795,200	Quên trừ giảm giá
56,237,920 (2573 dòng)	Không loại trạm DN07 — tổng tiền vẫn tính được nhưng 24 chuyến không có <code>ten_tram</code> , <code>thanh_pho</code>

Lỗi thường gặp khác: Chỉ bỏ khoảng trắng và thay `+84` → còn 68 số “không đủ 10 chữ số” (sốt dấu chấm, gạch ngang, đầu `84-`); bỏ ký tự không phải số nhưng quên đổi `84` → 71. Ép `so_dien_thoai` sang số nguyên làm mất số 0 ở đầu. Ghép trạm bằng `on='tram_muon'` → `KeyError` vì bảng `tram` không có cột này.

Phần D — Biến đổi & thu gọn dữ liệu

1,5 điểm

Câu D1. Biến đổi dữ liệu

0,75 điểm

Chỉ số	Kết quả chuẩn
Mode của <code>diem_dg</code>	4 (Hài lòng)
Phân bố <code>diem_dg</code> sau khi điền	1: 2 · 2: 95 · 3: 520 · 4: 1464 · 5: 468
Skew <code>phut</code> → <code>log_phut</code>	1.491 → 0.020
Số chuyến theo khung giờ	Sáng 718 · Trưa chiều 759 · Tan tầm 867 · Tối 205
Cột one-hot	<code>kg_Sáng</code> · <code>kg_Trưa chiều</code> · <code>kg_Tan tầm</code> · <code>kg_Tối</code>

```
THU_TU = {'Rất không hài lòng': 1, 'Không hài lòng': 2, 'Bình thường': 3,
          'Hài lòng': 4, 'Rất hài lòng': 5}
df['diem_dg'] = df['danh_gia'].map(THU_TU)
df['diem_dg'] = df['diem_dg'].fillna(df['diem_dg'].mode()[0]).astype(int)

df['log_phut'] = np.log1p(df['phut'])
st.skew(df['phut']), st.skew(df['log_phut'])
```

1.491, 0.020

```
df['khung_gio'] = pd.cut(df['bat_dau'].dt.hour, bins=[0, 9, 16, 20, 24], right=False,
                        labels=['Sáng', 'Trưa chiều', 'Tan tầm', 'Tối'])
pd.get_dummies(df['khung_gio'], prefix='kg', dtype=int).columns.tolist()
```

Tiêu chí	Điểm
Ảnh xạ đúng thứ tự 1–5, điền mode (mode() [0]), kiểu số nguyên	0,25
np.log1p (hoặc np.log(1 + x)), in skew trước và sau	0,2
pd.cut đúng mốc và right=False ; one-hot đúng 4 cột có tiền tố	0,3

Lỗi thường gặp: Quên right=False → các chuyến bắt đầu lúc 9h, 16h, 20h bị xếp lùi một khung: Sáng 825 · Trưa chiều 844 · Tan tầm 779 · Tối 101 — trừ 0,15. Dùng astype('category').cat.codes → mã theo thứ tự chữ cái (Bình thường 0, Hải lòng 1, Không hải lòng 2, ...) sai thứ bậc: trừ 0,2. Quên [0] sau mode() → phần lớn ô vẫn trống. Dùng np.log không cộng 1 vẫn chạy (skew -0.054) vì sau C2 mọi phut ≥ 2 — chấp nhận nhưng nhắc rủi ro.

Câu D2. Thu gọn dữ liệu theo khách hàng

0,75 điểm

Chỉ số	Kết quả chuẩn
Kích thước bảng khách	(474, 6) — trong 500 khách có 26 khách không có chuyến nào
Nhóm chi tiêu (qcut)	Thấp 120 · Trung bình 117 · Khá 118 · Cao 119 (474 không chia hết cho 4 và có giá trị trùng ở mép nhóm)
Cặp $ r > 0,7$	phut_tb – km_tb 0.91 · so_chuyen – tong_chi 0.78
Tỉ lệ phương sai tích lũy	0.508 · 0.725 · 0.885 · 0.973 · 0.988 · 1.000
Số thành phần PCA giữ ≥ 90%	4

```
thanh_vien = df[df['goi_cuoc'] != 'Khách lẻ']
khach = thanh_vien.groupby('ma_kh').agg(
    so_chuyen=('ma_cd', 'count'), tong_chi=('doanh_thu', 'sum'),
    phut_tb=('phut', 'mean'), km_tb=('quang_duong_km', 'mean'), diem_tb=('diem_dg', 'mean'),
    ti_le_xe_may=('loai_xe', lambda s: (s == 'Xe máy điện').mean()))
khach['tong_chi'] = khach['tong_chi'] / 1e3
khach['nhom_chi'] = pd.qcut(khach['tong_chi'], q=4,
                           labels=['Thấp', 'Trung bình', 'Khá', 'Cao'])

tq = khach.drop(columns='nhom_chi').corr()
Z = StandardScaler().fit_transform(khach.drop(columns='nhom_chi'))
PCA(n_components=0.9).fit(Z).n_components_ # 4
```

Giải thích được chấp nhận: km_tb gần như bằng tốc độ × phut_tb ÷ 60 — quãng đường tỉ lệ thuận với thời gian đi nên hai cột mang cùng thông tin về “độ dài chuyến”; tong_chi ≈ so_chuyen × cước trung bình mỗi chuyến nên phần lớn thông tin của tong_chi đã nằm trong số chuyến. Giữ một cột trong mỗi cặp là đủ cho phần lớn phân tích. PCA “gộp” được hai cặp này nên 6 cột còn 4 thành phần.

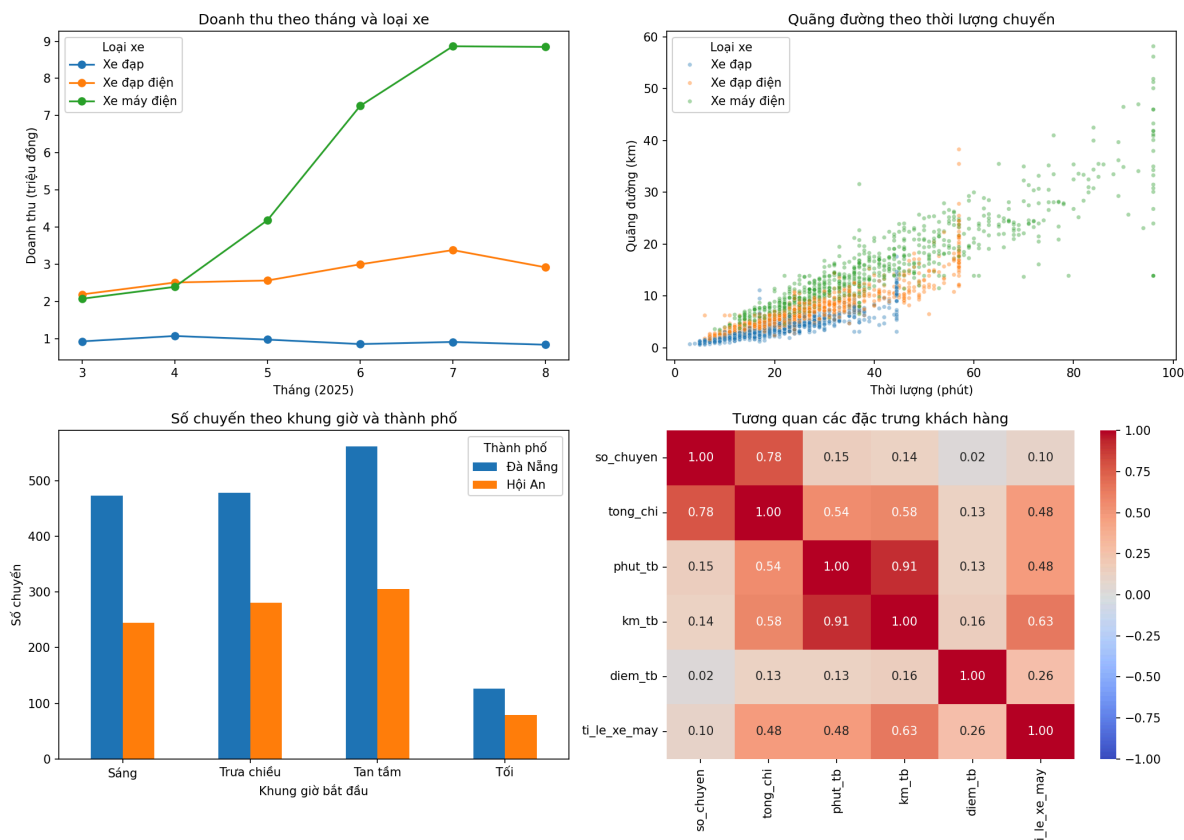
Tiêu chí	Điểm
groupby + agg ra đúng 6 cột, bỏ Khách lẻ, đơn vị nghìn đồng	0,25
pd.qcut 4 nhóm	0,1
Ma trận tương quan, chỉ ra đúng 2 cặp và giải thích dư thừa	0,2
Chuẩn hoá trước PCA, ra 4 thành phần (sklearn hoặc tự tính trị riêng bằng numpy)	0,2

Lỗi thường gặp: Không chuẩn hoá trước PCA → `tong_chi` (phương sai 4806) lấn át các cột khác (`ti_le_xe_may` chỉ 0,09), PC1 đã giữ 98,0% → ra 1 thành phần: phần PCA 0,05. Đọc nhầm 3 thành phần (88,5%) là “đủ 90%” → trừ 0,1. Dùng `pd.cut` thay `qcut` → bốn nhóm cùng độ rộng nhưng lệch số lượng ($385 \cdot 67 \cdot 17 \cdot 5$): không có điểm ý `qcut`. Để `tong_chi` ở đơn vị đồng không ảnh hưởng tương quan và PCA đã chuẩn hoá — chỉ trừ 0,05 ở ý đơn vị.

Phần E — Trực quan hoá

2,0 điểm

Hình mẫu: `dap_an/bieu_do.png`.



Hình kết quả mẫu Phần E — lưu với dpi = 150

Tháng	3	4	5	6	7	8
Xe đạp (triệu)	0.9	1.1	1.0	0.8	0.9	0.8
Xe đạp điện (triệu)	2.2	2.5	2.6	3.0	3.4	2.9
Xe máy điện (triệu)	2.1	2.4	4.2	7.3	8.9	8.8

Số chuyến	Sáng	Trưa chiều	Tan tầm	Tối
Đà Nẵng	473	478	562	126
Hội An	245	281	305	79

```
fig, axes = plt.subplots(2, 2, figsize=(14, 10))
THU_TU_XE = ['Xe đạp', 'Xe đạp điện', 'Xe máy điện']

theo_thang = (df.groupby([df['bat_dau'].dt.month, 'loai_xe'])['doanh_thu'].sum() / 1e6
               ).unstack()[THU_TU_XE]
theo_thang.plot(ax=axes[0, 0], marker='o')
```

```

axes[0, 0].set(title='Doanh thu theo tháng và loại xe', xlabel='Tháng (2025)',
              ylabel='Doanh thu (triệu đồng)')

sns.scatterplot(data=df, x='phut', y='quang_duong_km', hue='loai_xe', hue_order=THU_TU_XE,
               alpha=0.4, s=12, ax=axes[0, 1])
axes[0, 1].set(title='Quãng đường theo thời lượng chuyển', xlabel='Thời lượng (phút)',
              ylabel='Quãng đường (km)')

kg = pd.crosstab(df['khung_gio'], df['thanh_pho']) # category → giữ thứ tự Sáng → Tối
kg.plot(kind='bar', ax=axes[1, 0], rot=0)
axes[1, 0].set(title='Số chuyến theo khung giờ và thành phố', xlabel='Khung giờ bắt đầu',
              ylabel='Số chuyến')

sns.heatmap(tq, annot=True, fmt='.2f', cmap='coolwarm', vmin=-1, vmax=1, ax=axes[1, 1])
axes[1, 1].set_title('Tương quan các đặc trưng khách hàng')

fig.tight_layout()
fig.savefig('bieu_do.png', dpi=150) # lưu TRƯỚC plt.show()
plt.show()

```

Tiêu chí	Điểm
Một hình, 2 hàng × 2 cột (<code>plt.subplots(2, 2)</code>), mỗi biểu đồ vẽ đúng ô bằng <code>ax=</code>	0,2
Biểu đồ đường: 3 loại xe, có marker và chú thích, trục tháng 3–8	0,35
Biểu đồ phân tán <code>phut × quang_duong_km</code> tô màu theo loại xe	0,3
Cột ghép khung giờ × thành phố, không chồng , thứ tự Sáng → Tối	0,35
Heatmap có số trên ô, thang màu khoá -1 đến 1	0,3
Tiêu đề và nhãn trục có đơn vị — cả bốn biểu đồ	0,2
Lưu <code>bieu_do.png</code> , <code>dpi=150</code>	0,1
Hai câu nhận xét hợp lý	0,2

Nhận xét được chấp nhận: (1) doanh thu Xe máy điện tăng mạnh từ 2,1 triệu (tháng 3) lên 8,9 triệu (tháng 7), vượt Xe đạp điện từ tháng 5 và gấp khoảng 3 lần vào tháng 8, trong khi Xe đạp điện chỉ nhích nhẹ và Xe đạp đi ngang quanh 1 triệu/tháng (tỉ trọng chuyển xe máy điện tăng từ 13% lên 39%); (2) các điểm xếp thành những dải thẳng đi qua gốc toạ độ — quãng đường tỉ lệ thuận với thời lượng — với độ dốc khác nhau: xe máy điện nhanh nhất (≈ 23 km/h), xe đạp điện ≈ 16 km/h, xe đạp ≈ 11 km/h; điều này giải thích tương quan 0,91 giữa `phut_tb` và `km_tb` ở D2. Chấp nhận thêm: các cột điểm dựng đứng ở $44,5 \cdot 57 \cdot 96$ phút là dấu vết winsorization của C2; Tan tằm là khung đông nhất ở cả hai thành phố, Đà Nẵng gần gấp đôi Hội An ở mọi khung giờ.

Lỗi thường gặp: Gọi `savefig` sau `plt.show()` → ảnh trắng, mất 0,1. Vẽ bằng `plt.plot` thay vì `ax=` khiến các biểu đồ đè lên ô cuối cùng: phân bố cục 0. `khung_gio` bị đổi sang chuỗi (hoặc tự tạo bằng `if/else`) → `crosstab` xếp theo chữ cái Sáng, Tan tằm, Trưa chiều, Tối — trừ 0,1. Dùng `stacked=True` → trừ 0,15. Heatmap không đặt `vmin=-1, vmax=1` → thang màu co về 0,02...1, mọi ô đều đỏ đậm dù tương quan yếu: trừ 0,1.

— HẾT —

Đáp án gồm lời giải tham khảo; mọi cách làm đúng khác đều được chấp nhận.